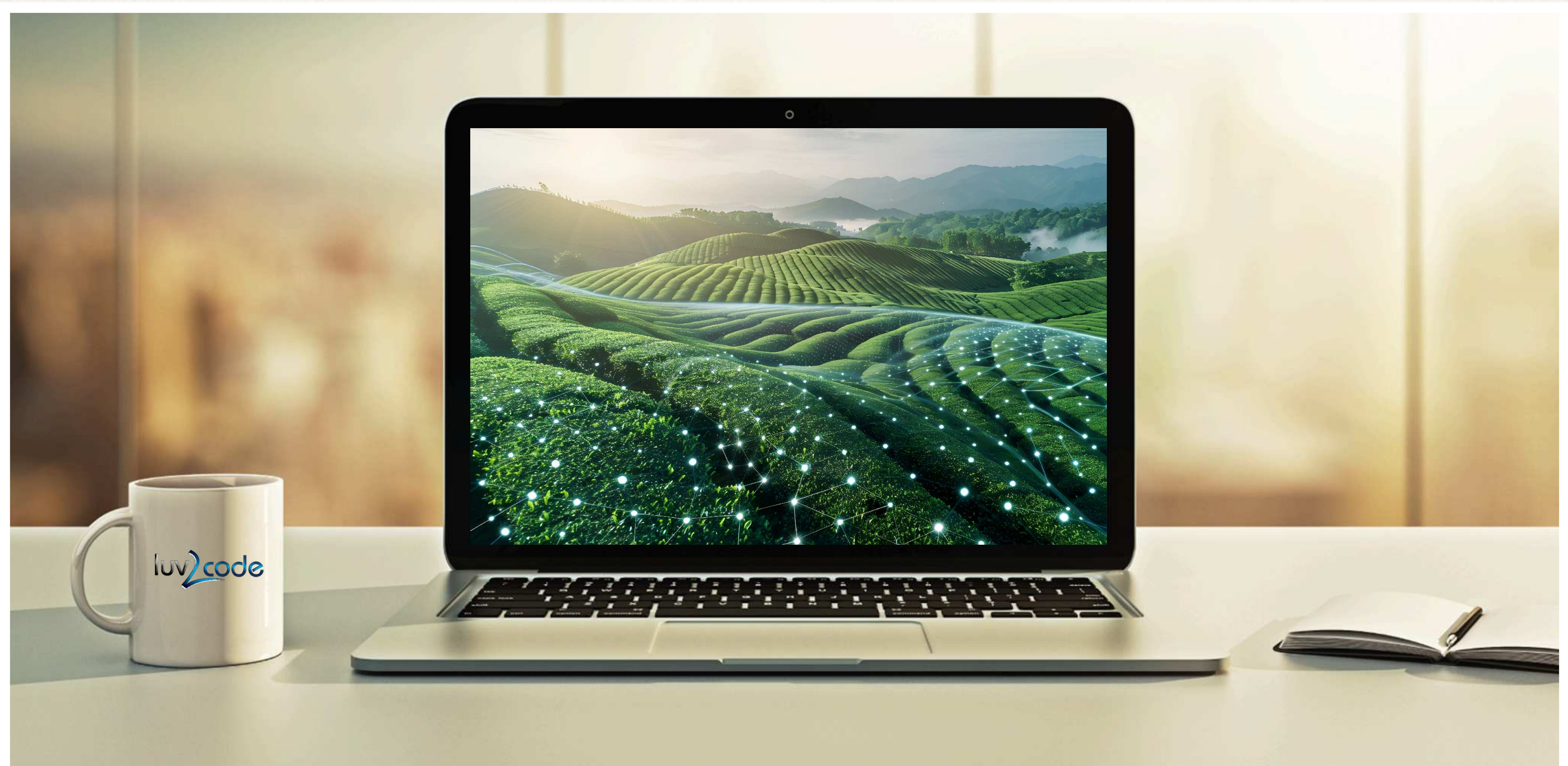


Project 3 Overview



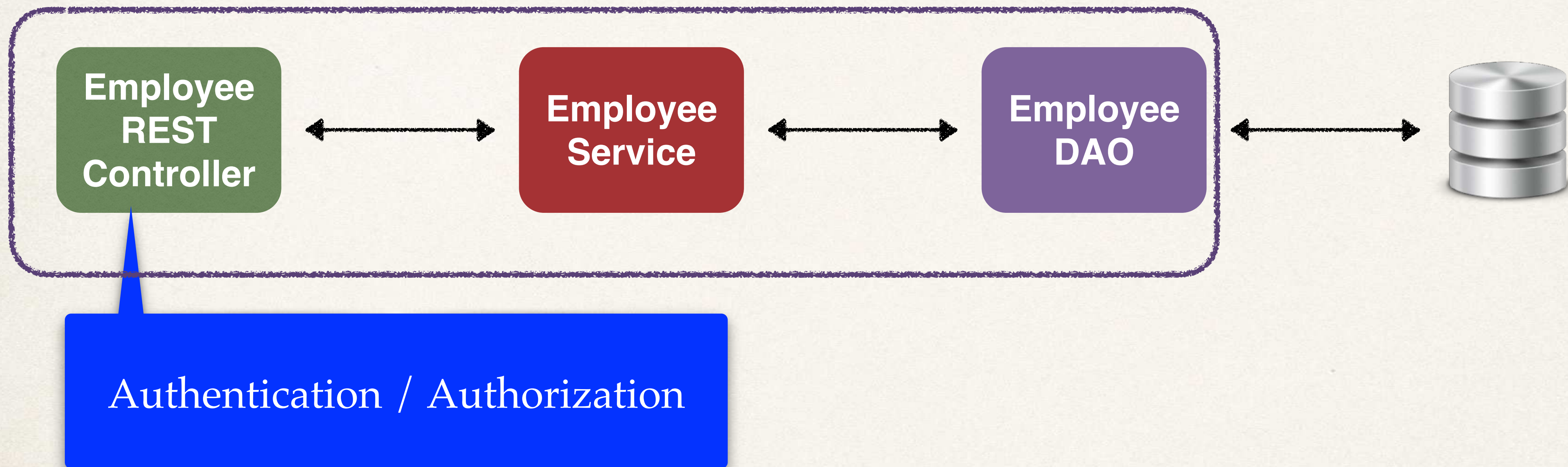
Project 3

- Project three we will be switching our focus to Employees instead of Books
- New Information will include:
 - SQL Databases
 - Authentication
 - Authorization
 - Hashing Passwords

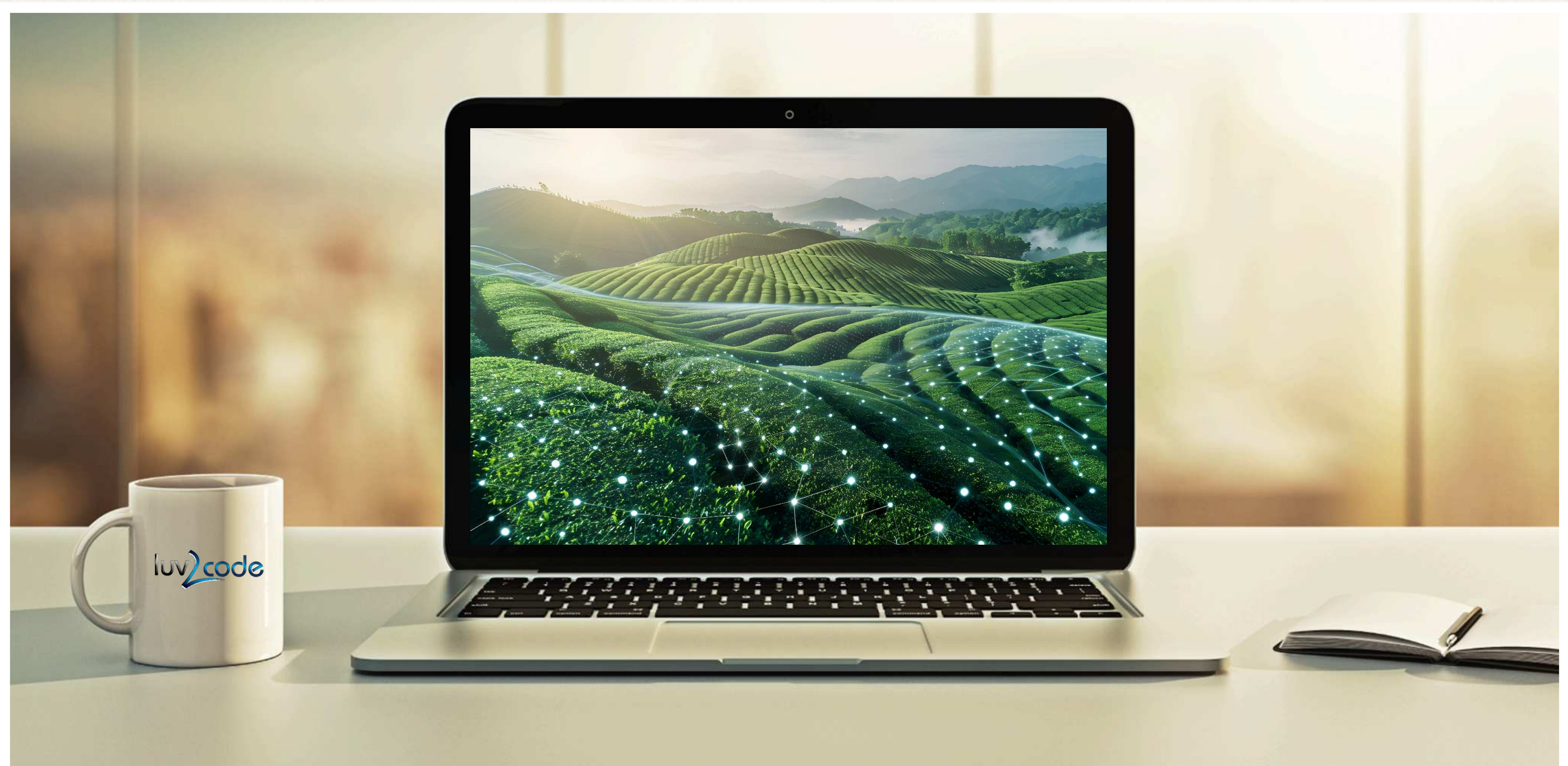
**Continued learning from
other projects**

Application Architecture

Spring Boot Security

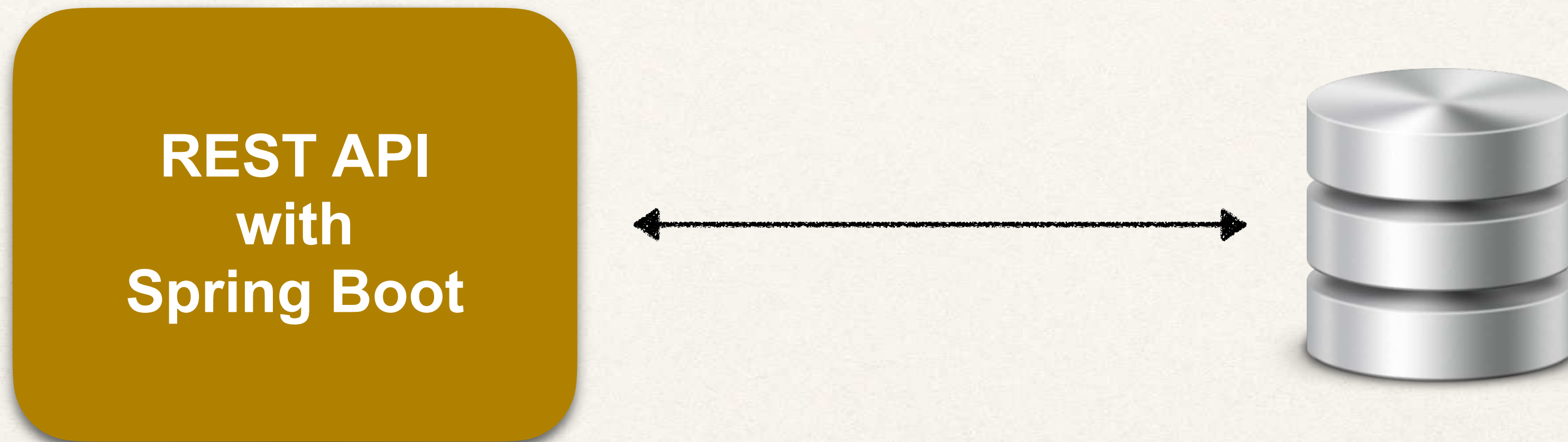


Spring Boot REST API - Real Time Project



Real-Time Project

REST API with Spring Boot that connects to a database



API Requirements

From the Boss

Create a REST API for the Employee Directory

REST clients should be able to

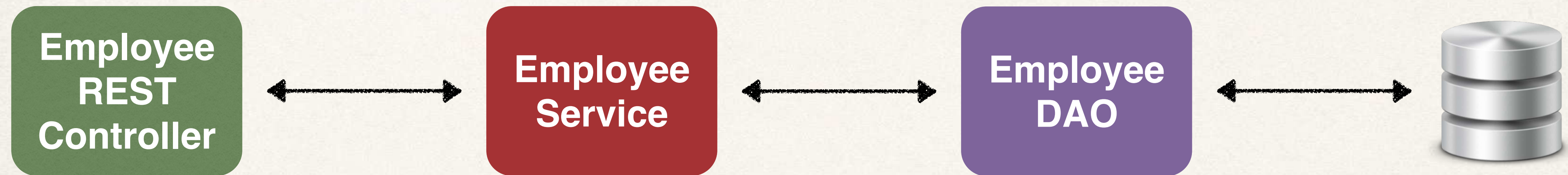
- Get a list of employees
- Get a single employee by id
- Add a new employee
- Update an employee
- Delete an employee

Development Process

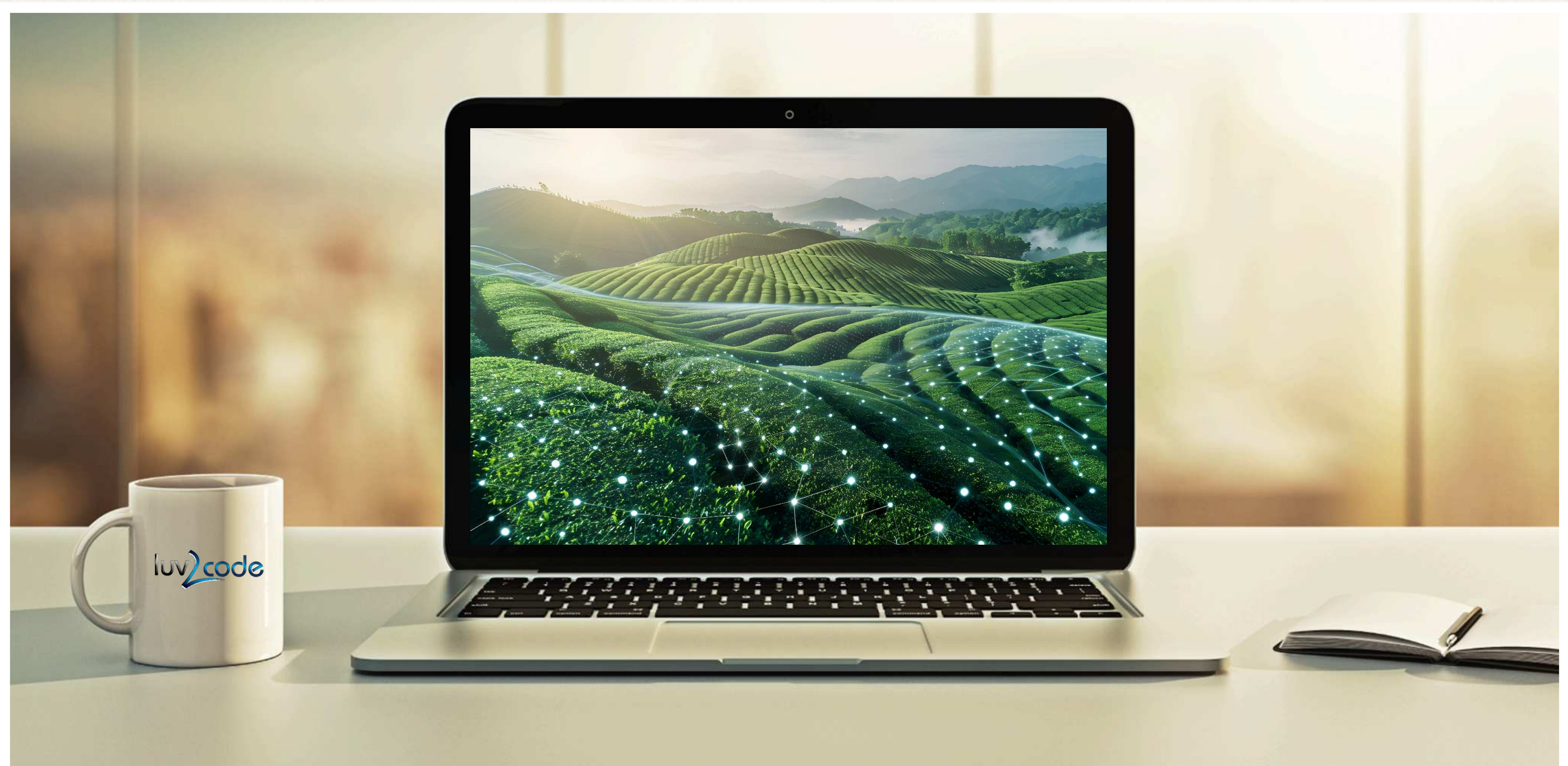
Step-By-Step

1. Create Spring Boot project using Spring Initializer
2. Set up Database Dev Environment
3. Get list of employees
4. Get single employee by ID
5. Add a new employee
6. Update an existing employee
7. Delete an existing employee

Application Architecture



Creating Spring Boot Project



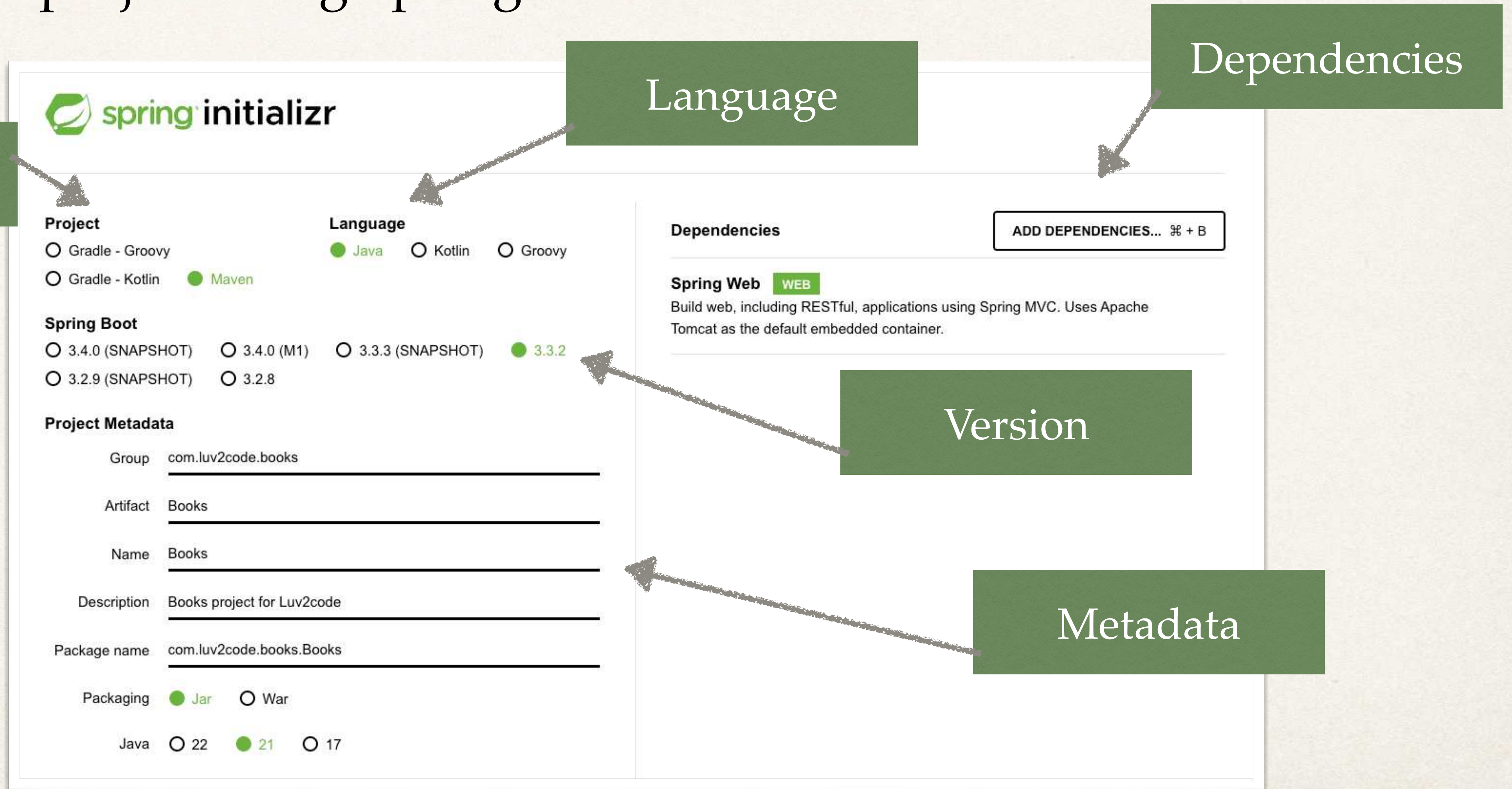
Spring Initializr

- Here we can set the project configurations for our upcoming Spring project.
- Allows us to choose the project, programming language, Spring Boot version, dependencies and metadata for our application.

start.spring.io

Spring Initializr

- Setup our project using Spring Initializr



The image shows the Spring Initializr web form with several green callout boxes and arrows pointing to specific fields:

- Project (maven)**: Points to the **Project** section where **Maven** is selected.
- Language**: Points to the **Language** section where **Java** is selected.
- Dependencies**: Points to the **Dependencies** section where **Spring Web** is selected.
- Version**: Points to the **Spring Boot** version selection where **3.3.2** is selected.
- Metadata**: Points to the **Project Metadata** section.

The form fields are as follows:

- Project**: ☐ Gradle - Groovy, ☐ Gradle - Kotlin, ☒ Maven
- Language**: ☒ Java, ☐ Kotlin, ☐ Groovy
- Spring Boot**: ☐ 3.4.0 (SNAPSHOT), ☐ 3.4.0 (M1), ☐ 3.3.3 (SNAPSHOT), ☒ 3.3.2, ☐ 3.2.9 (SNAPSHOT), ☐ 3.2.8
- Project Metadata**:
 - Group:
 - Artifact:
 - Name:
 - Description:
 - Package name:
 - Packaging: ☒ Jar, ☐ War
 - Java: ☐ 22, ☒ 21, ☐ 17
- Dependencies**:
 - Spring Web** ☒ WEB
 - Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Initializr

- Spring Initializr will download a .zip file onto our machine.
- We will need to unzip the .zip file.
- Open the project on IntelliJ.
- Download all dependencies using Maven.
- Sounds fun - Let's go ahead and dive in!

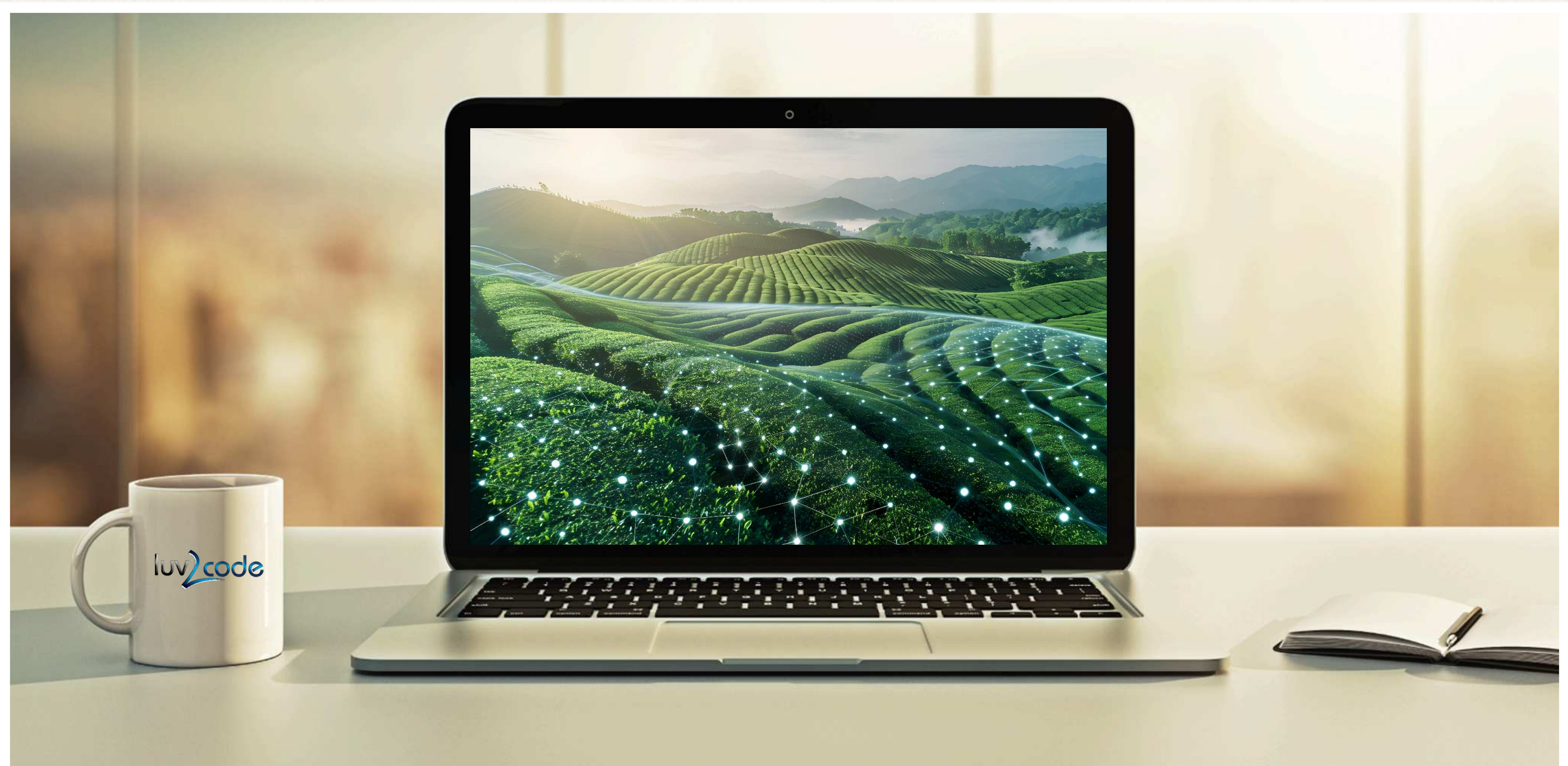
Development Process

Step-By-Step

1. Create Spring Boot project using Spring Initializer
2. Set up Database Dev Environment
3. Get list of employees
4. Get single employee by ID
5. Add a new employee
6. Update an existing employee
7. Delete an existing employee

Our Task

H2 Database Overview



What is a H2 database?

- H2 is a lightweight, open-source RDBMS written in Java.
- Most used as a memory or file database.
- Memory - On RAM, usually used for testing.
- File - Persistent database where it stores data on a file within our project directory.
- We will be using a file database configuration for this project.

How do we use a H2 database?

- We already have H2 installed within our POM file:

```
<dependency>  
  <groupId>com.h2database</groupId>  
  <artifactId>h2</artifactId>  
  <scope>runtime</scope>  
</dependency>
```

- This is all the installation we need to properly use a H2 database!

How do we use a H2 database?

- We then need to add the configurations in our application.properties file.

```
# Configs
spring.datasource.url=jdbc:h2:file:./data/employeedb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

# Console
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.jpa.hibernate.ddl-auto=update
```


employee.sql

1. File: employee SQL table.
2. Create a new database table: **employee**.
3. Load table with sample data.



SQL Scripts available in resources

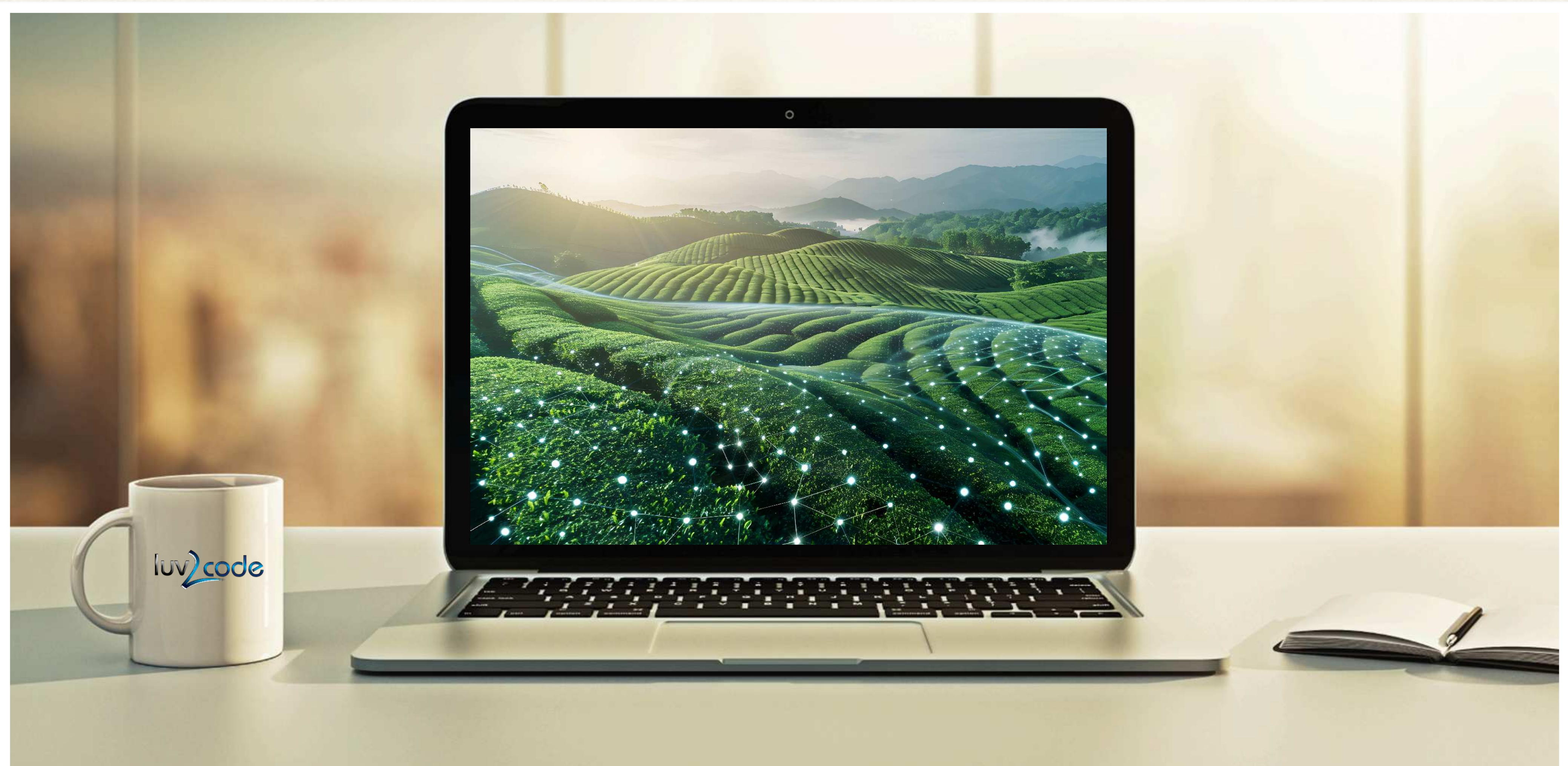
Create Employee Entity

- Create the Employee entity that matches the database table

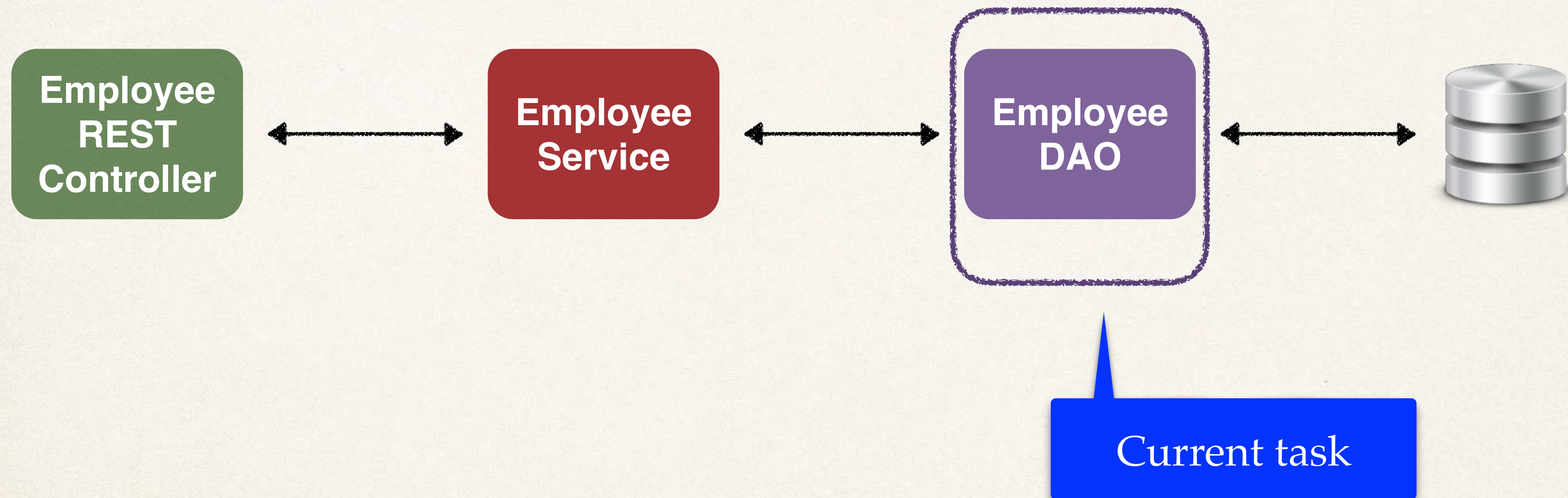
```
public class Employee {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    @Column(name="id")  
    private long id;  
  
    @Column(name="first_name")  
    private String firstName;  
  
    @Column(name="last_name")  
    private String lastName;  
  
    @Column(name="email")  
    private String email;
```

< Constructor, Getters & Setters >

Create DAO in Spring Boot



Application Architecture



Development Process

1. Create Spring Boot project using Spring Initializer
2. Set up Database Dev Environment
3. Get list of employees
4. Get single employee by ID
5. Add a new employee
6. Update an existing employee
7. Delete an existing employee

Step-By-Step

**Let's build a
DAO layer for this**

DAO Interface

```
public interface EmployeeDAO {  
  
    List<Employee> findAll();  
  
}
```


DAO Impl

Same interface for
consistent API

```
@Repository
public class EmployeeDAOJpaImpl implements EmployeeDAO {

    private EntityManager entityManager;

    @Autowired
    public EmployeeDAOJpaImpl(EntityManager theEntityManager) {
        entityManager = theEntityManager;
    }

    ...
}
```

Automatically created
by Spring Boot

Constructor
injection

Get a list of employees

```
@Override
public List<Employee> findAll() {

    // create a query
    TypedQuery<Employee> theQuery =
        entityManager.createQuery("from Employee", Employee.class);

    // execute query and get result list
    List<Employee> employees = theQuery.getResultList();

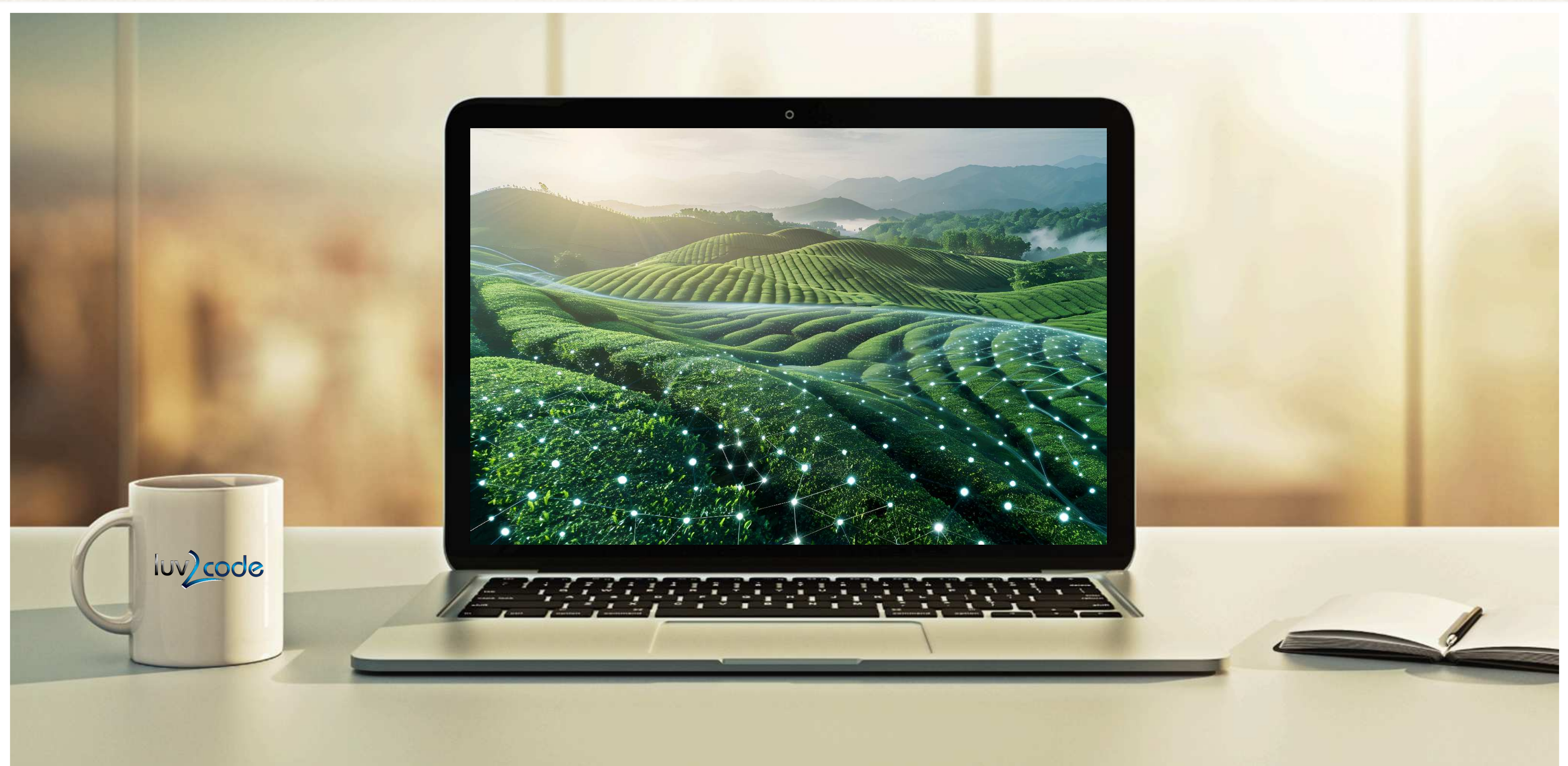
    // return the results
    return employees;
}
```


Development Process

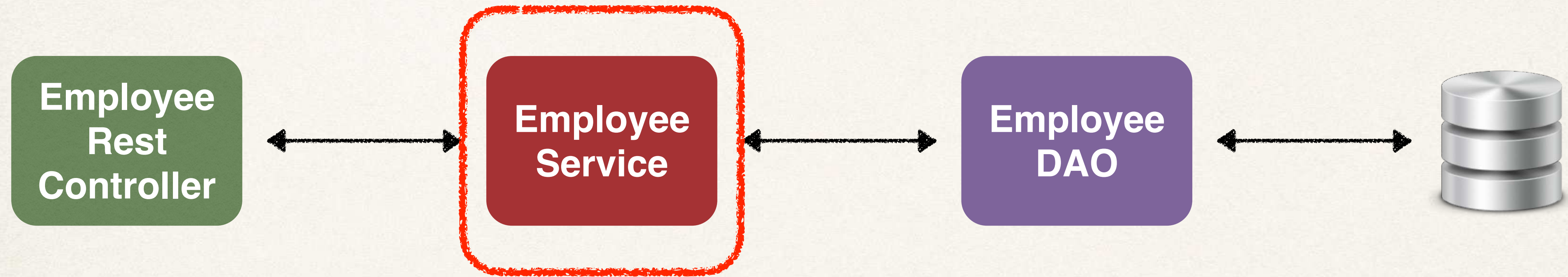
Step-By-Step

1. Update db configs in application.properties
2. Create Employee entity
3. Create DAO interface
4. Create DAO implementation
5. Create REST controller to use DAO

Define Services with @Service

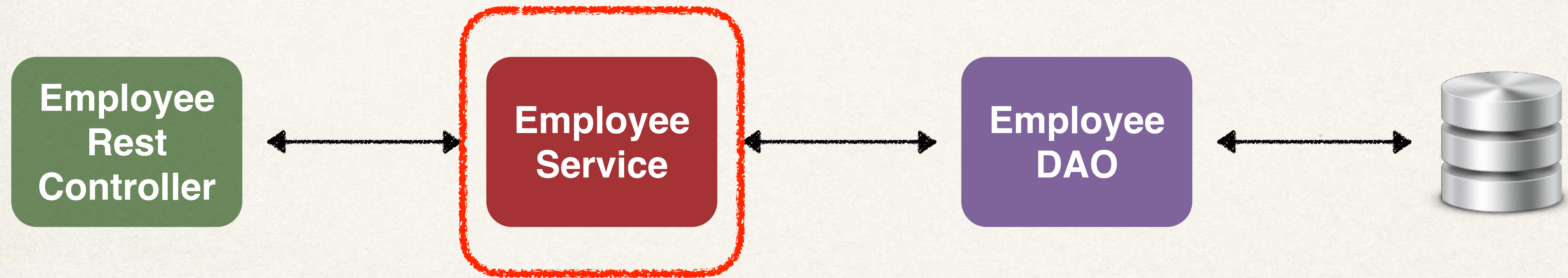


Refactor: Add a Service Layer

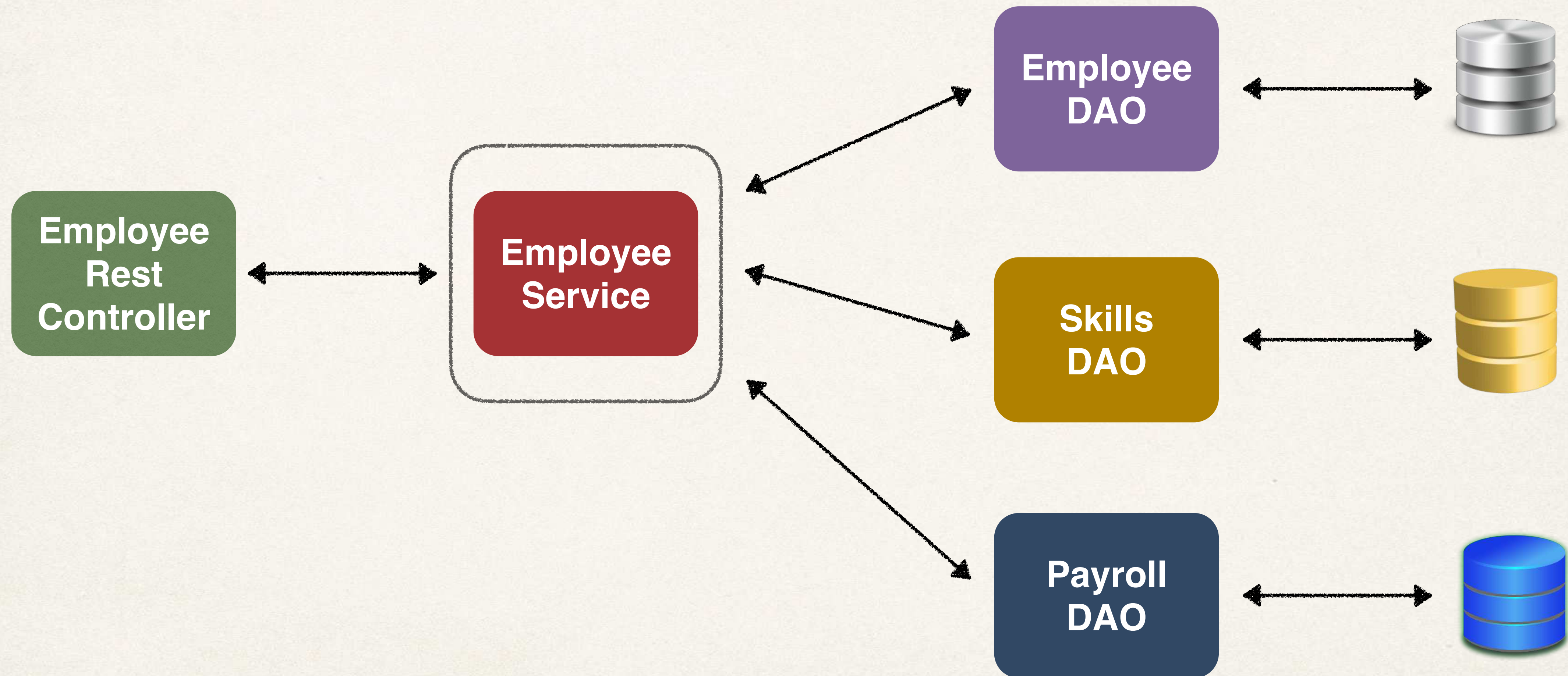


Purpose of Service Layer

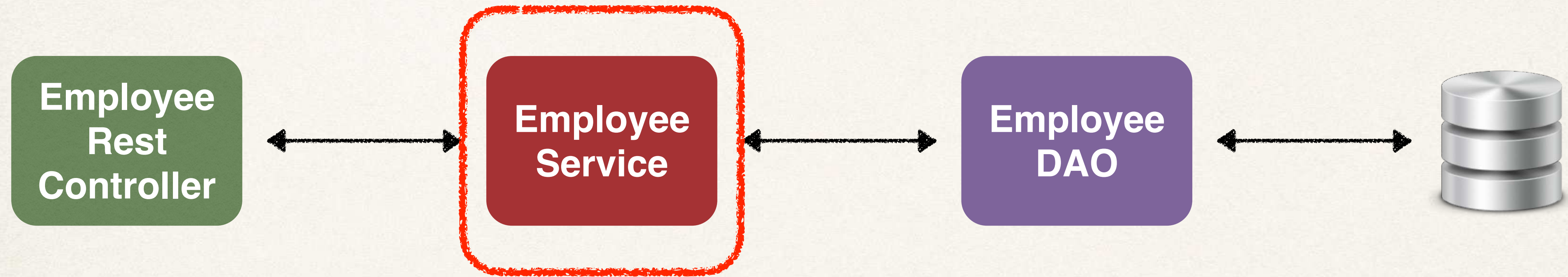
- ❖ **Service Facade** design pattern
- ❖ Intermediate layer for custom business logic
- ❖ Integrate data from multiple sources (DAO / repositories)



Integrate Multiple Data Sources

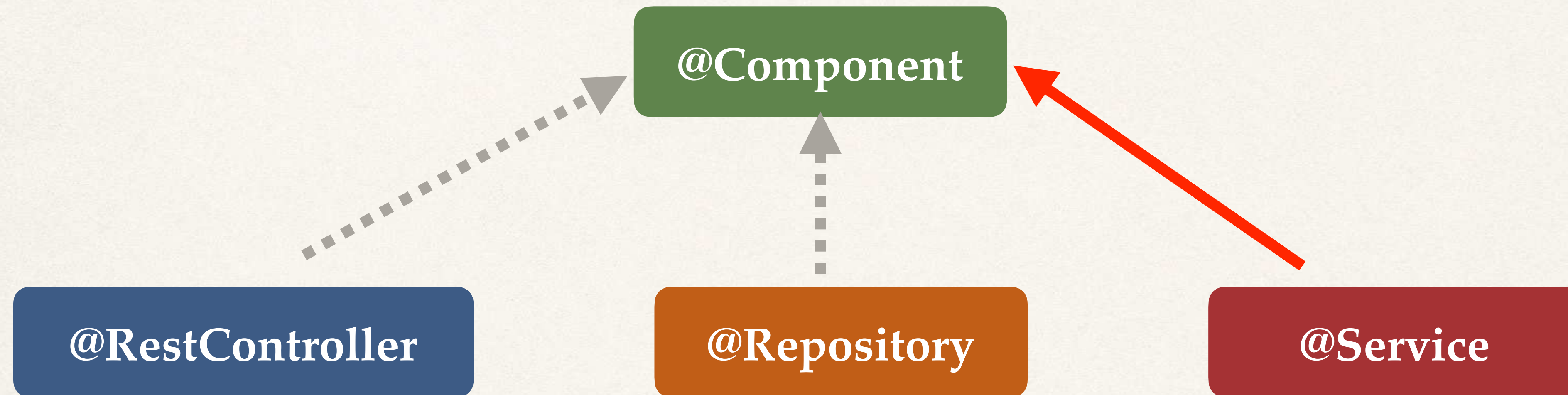


Most Times - Delegate Calls



Specialized Annotation for Services

- Spring provides the **@Service** annotation



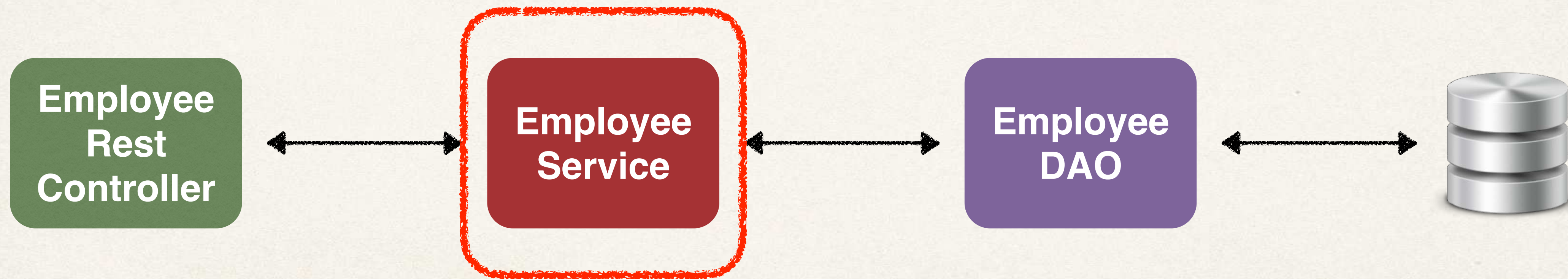
Specialized Annotation for Services

- **@Service** applied to Service implementations
- Spring will automatically register the Service implementation
 - thanks to component-scanning

Employee Service

1. Define Service interface
2. Define Service implementation
 - Inject the EmployeeDAO

Step-By-Step



Step 1: Define Service interface

```
public interface EmployeeService {  
  
    List<Employee> findAll();  
  
}
```


Step 2: Define Service implementation

@Service - enables component scanning

@Service

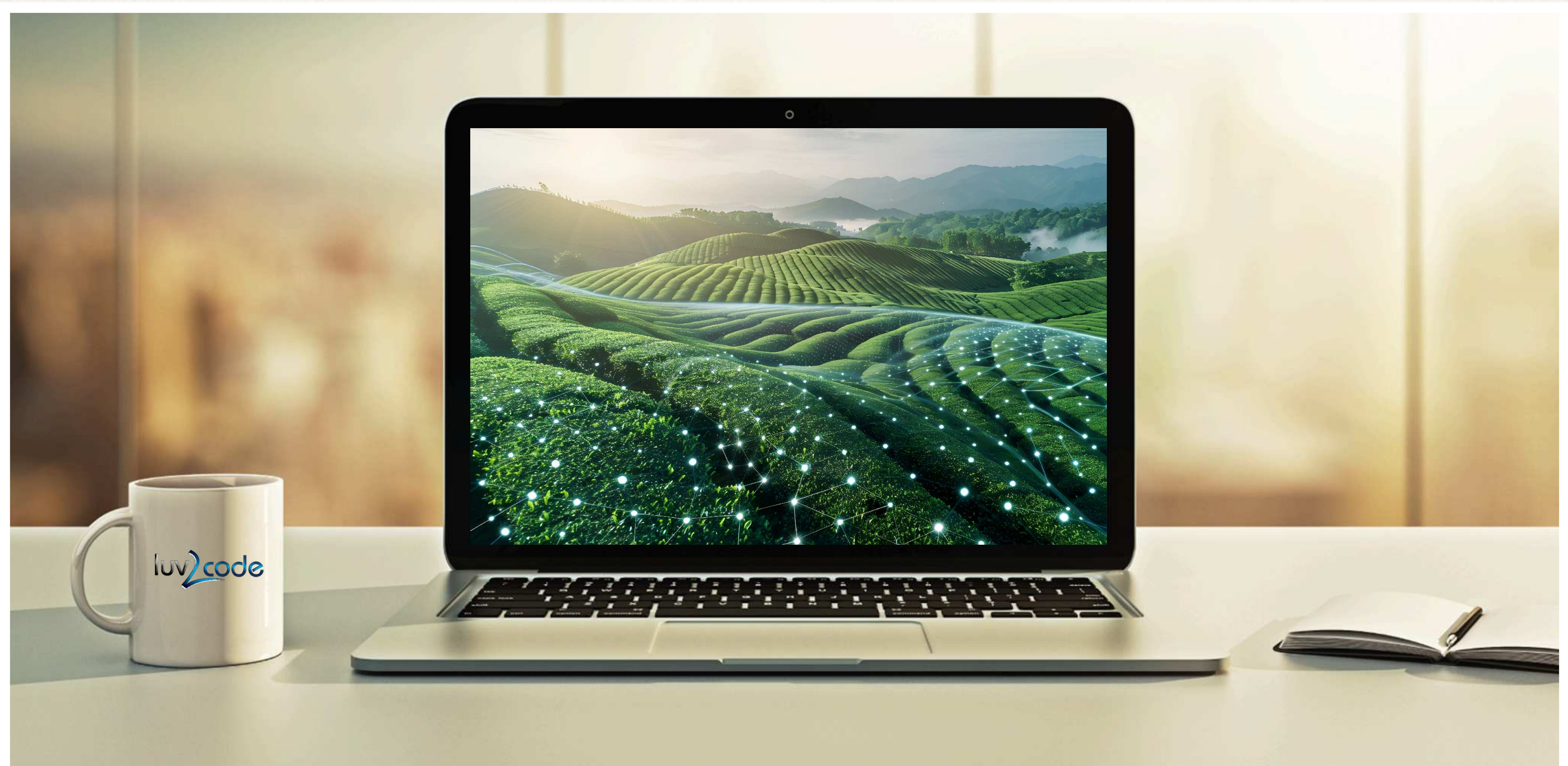
```
public class EmployeeServiceImpl implements EmployeeService {
```

```
// inject EmployeeDAO ...
```

@Override

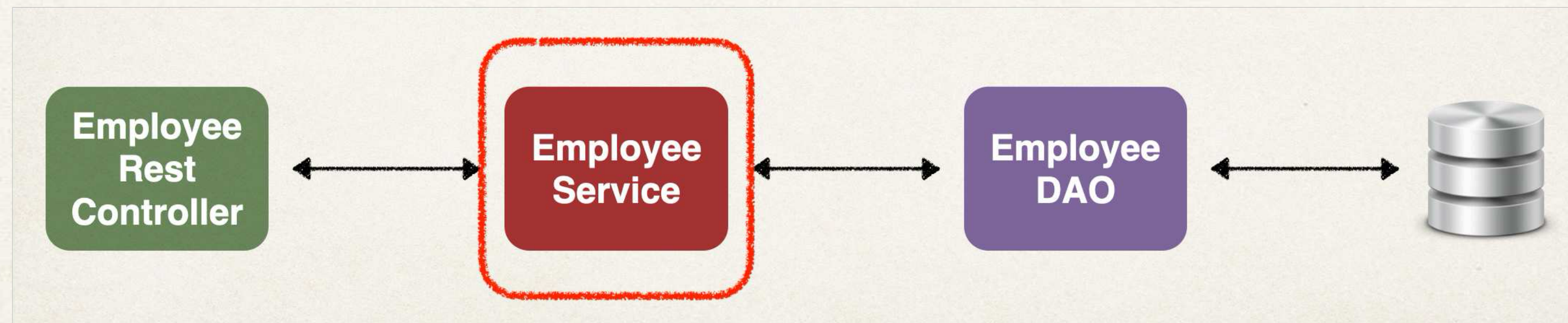
```
public List<Employee> findAll() {  
    return employeeDAO.findAll();  
}  
}
```


DAO: Find, Add, Update and Delete



Service Layer - Best Practice

- ❖ Best practice is to apply transactional boundaries at the service layer
- ❖ It is the service layer's responsibility to manage transaction boundaries
- ❖ For implementation code
 - ❖ Apply @Transactional on service methods
 - ❖ Remove @Transactional on DAO methods if they already exist

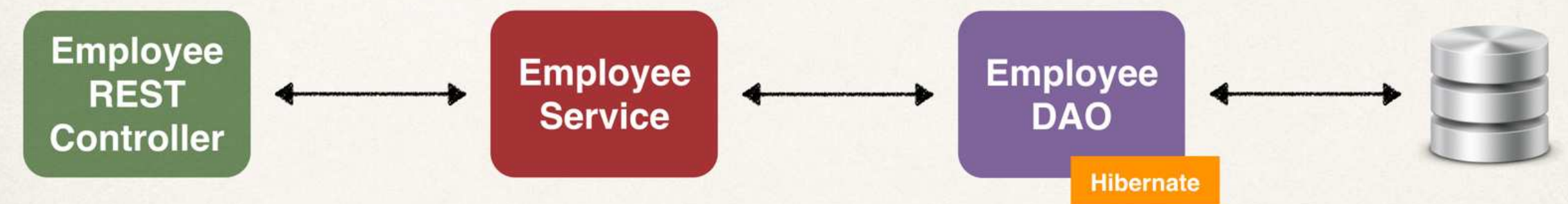


Development Process

Step-By-Step

1. Create Spring Boot project using Spring Initializer
2. Set up Database Dev Environment
3. Get list of employees
4. Get single employee by ID
5. Add a new employee
6. Update an existing employee
7. Delete an existing employee

DAO methods



DAO: Get a single employee

```
@Override
public Employee findById(int theId) {

    // get employee
    Employee theEmployee = entityManager.find(Employee.class, theId);

    // return employee
    return theEmployee;
}
```



DAO: Add or Update employee

Note: We don't use @Transactional at DAO layer
It will be handled at Service layer

```
@Override
public Employee save(Employee theEmployee) {

    // save or update the employee
    Employee dbEmployee = entityManager.merge(theEmployee);

    // return dbEmployee
    return dbEmployee;
}
```

if id == 0
then save/insert
else update

Return dbEmployee
It has updated id from the database
(in the case of insert)

DAO: Delete an existing employee

Note: We don't use @Transactional at DAO layer
It will be handled at Service layer

```
@Override
public void deleteById(int theId) {

    // find the employee by id
    Employee theEmployee = entityManager.find(Employee.class, theId);

    // delete the employee
    entityManager.remove(theEmployee);
}
```



Service: Find, Add, Update and Delete

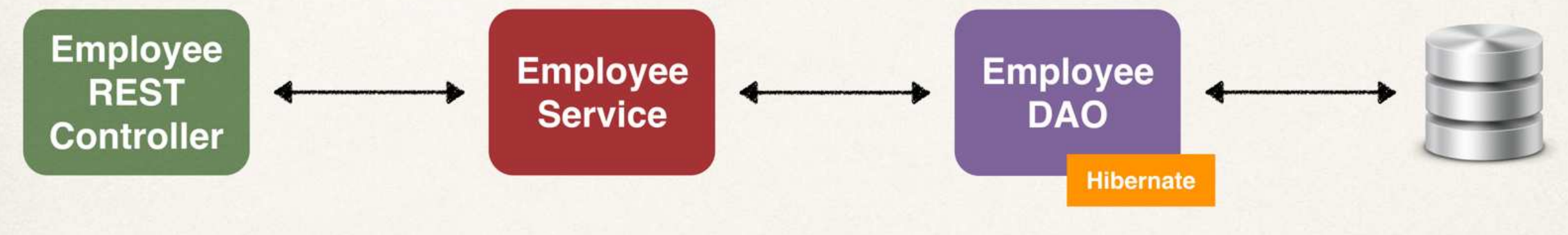


Development Process

Step-By-Step

1. Create Spring Boot project using Spring Initializer
2. Set up Database Dev Environment
3. Get list of employees
4. Get single employee by ID
5. Add a new employee
6. Update an existing employee
7. Delete an existing employee

Service methods



Create EmployeeRequest Entity

- Request that comes from client for validation

```
public class EmployeeRequest {
```

```
    @NotBlank  
    @Size(min=2, max=50)  
    private String firstName;
```

```
    @NotBlank  
    @Size(min=2, max=50)  
    private String lastName;
```

```
    @NotBlank  
    @Email  
    private String email;
```

```
< Constructor, Getters & Setters >
```

We will start with no data validation and then add in future lecture.

EmployeeRequest Conversion

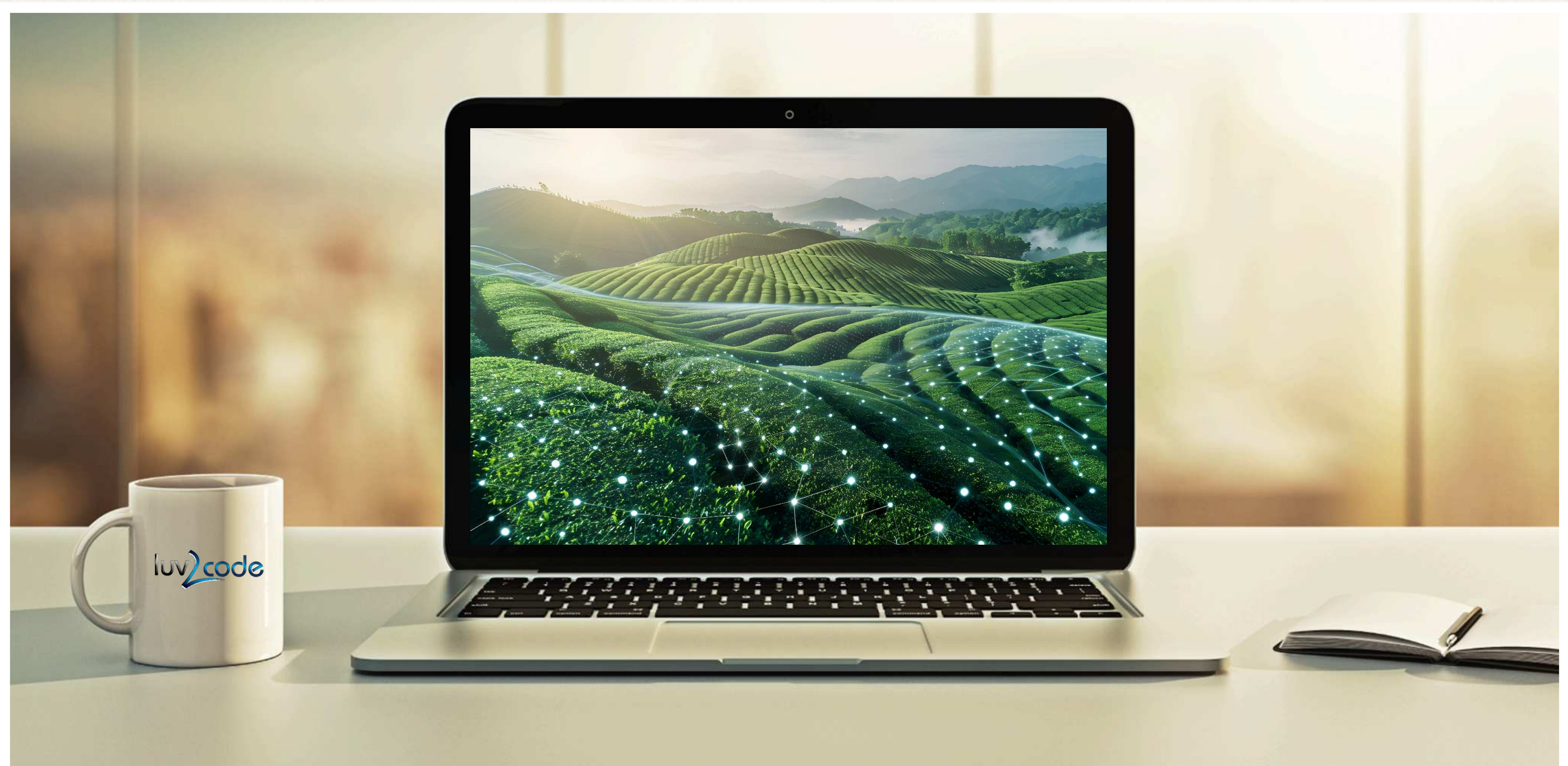
- Convert EmployeeRequest to Employee in Service Layer.

```
public Employee save(EmployeeRequest employeeRequest) {  
    Employee employee = convertToEmployee(0, employeeRequest);  
    return employeeRepository.save(employee);  
}
```

- convertToEmployee logic:

```
public Employee convertToEmployee(long id, EmployeeRequest employeeRequest) {  
    return new Employee(id, employeeRequest.getFirstName(),  
        employeeRequest.getLastName ...);  
}
```


Swagger Implementation

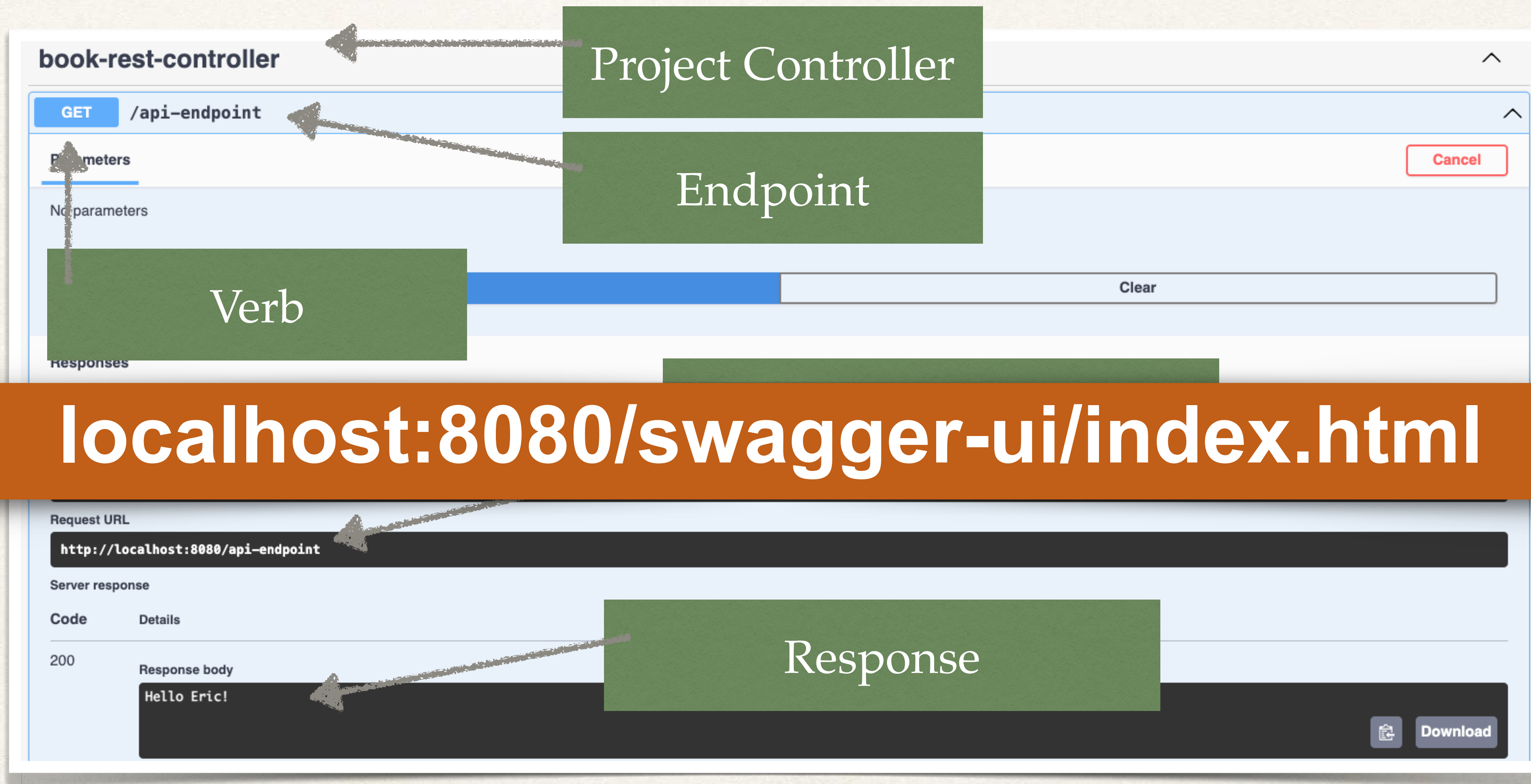


What is Swagger?

- Swagger is a set of tools and specifications for designing, building, documenting and consuming RESTful web services.
- Open-source project for streamlining API development by providing a common framework and API descriptions.
- Swagger UI implementation will allow us to be able to document and call our API endpoints right from our application.
- No third party tools will be needed!
- Win-win solution for implement Swagger within our application.

What is Swagger?

- Instead of going straight to the API endpoint, we can go to Swaggers.



What is Swagger?

- Path is hard to remember:

- localhost:8080/swagger-ui/index.html

Too long to remember

- We can fix this to redirect based off of /docs.

- New endpoint for Swagger: localhost:8080/docs

Much better :-)

File: application.properties

```
springdoc.swagger-ui.path=/docs
```


Create EmployeeRequest Entity

- Request that comes from client for validation

```
public class EmployeeRequest {
```

```
    @NotBlank  
    @Size(min=2, max=50)  
    private String firstName;
```

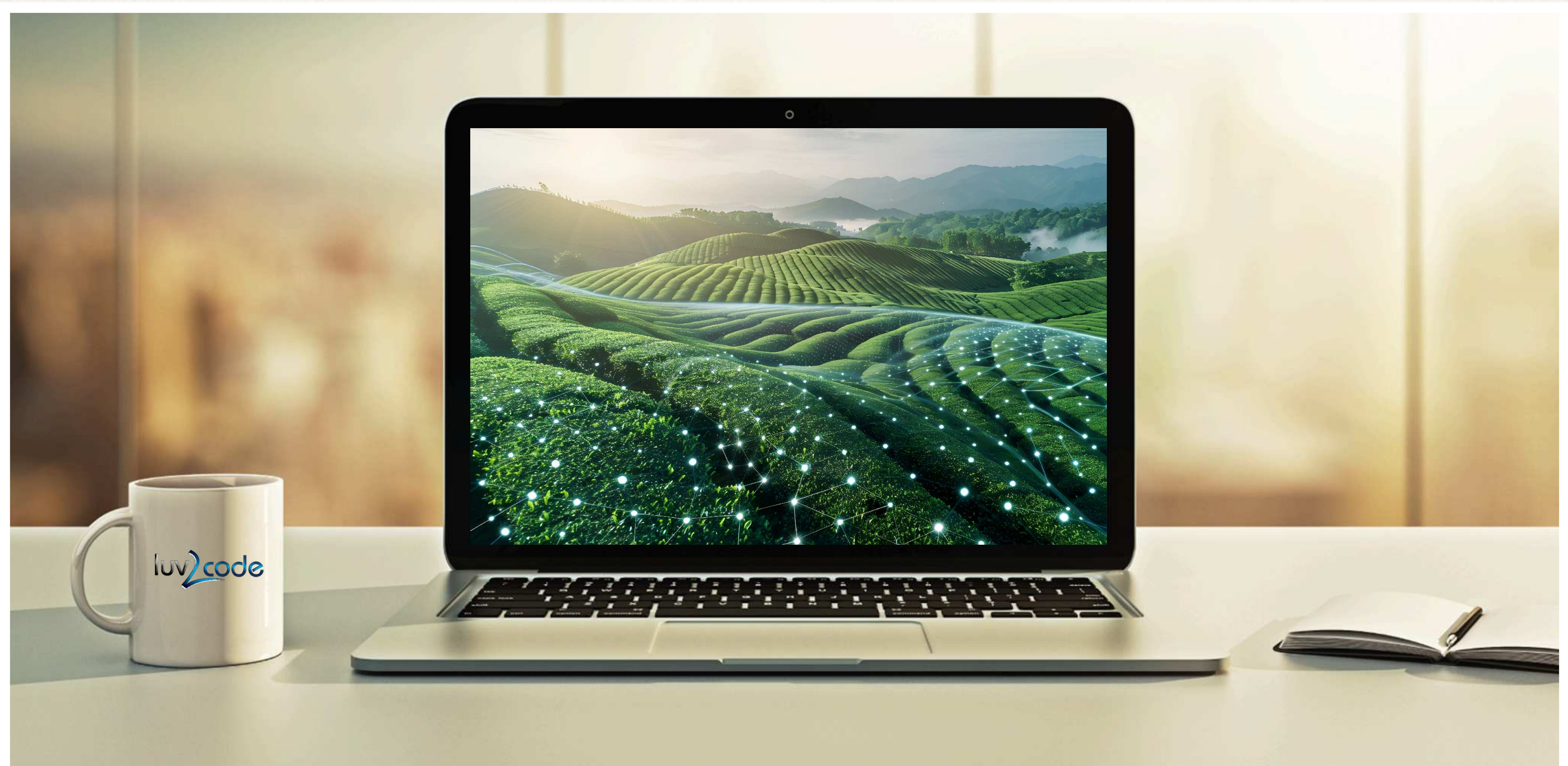
```
    @NotBlank  
    @Size(min=2, max=50)  
    private String lastName;
```

```
    @NotBlank  
    @Email  
    private String email;
```

```
< Constructor, Getters & Setters >
```

Add data validation to our EmployeeRequest object.

Rest Controller Methods - Find

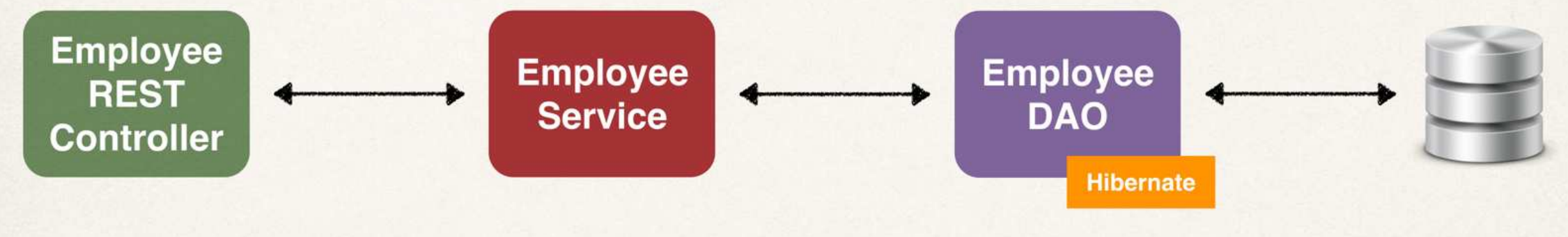


Development Process

Step-By-Step

1. Create Spring Boot project using Spring Initializer
2. Set up Database Dev Environment
3. Get list of employees
4. Get single employee by ID
5. Add a new employee
6. Update an existing employee
7. Delete an existing employee

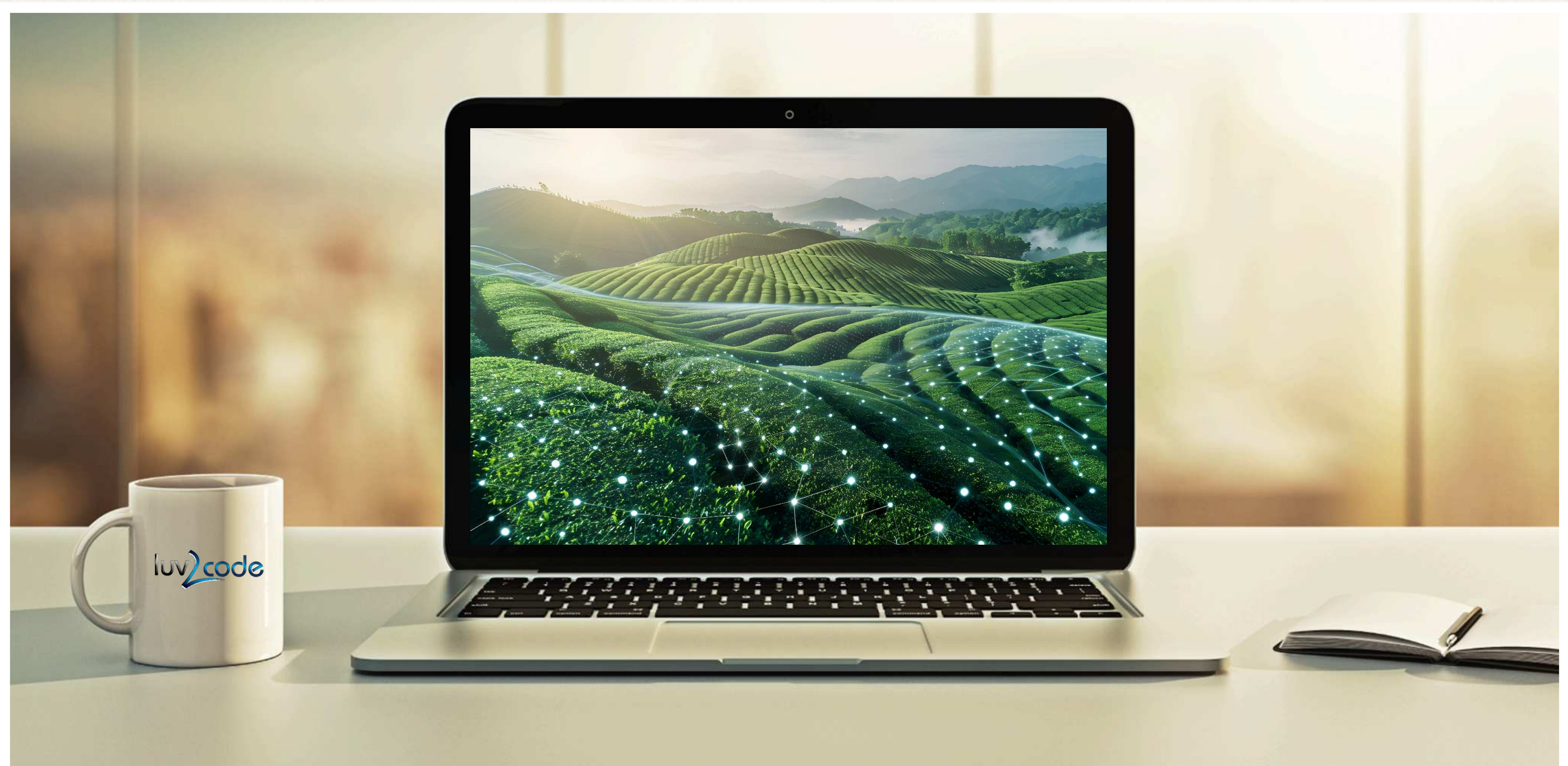
Rest Controller
methods



Read a Single Employee



Rest Controller Methods - Create



EmployeeRequest Validation

- Add @Valid to create the validation on the RequestBody.

```
public Employee addEmployee(@Valid @RequestBody EmployeeRequest theEmployee) {  
    Employee dbEmployee = employeeService.save(theEmployee)  
    return dbEmployee;  
}
```



- Convert EmployeeRequest into Employee in Service layer.

```
@ResponseStatus(HttpStatus.CREATED)  
@PostMapping()  
public Employee addEmployee(@Valid @RequestBody EmployeeRequest theEmployee)  
{}
```


Create a New Employee

Since new employee,
we are not
passing id / primary key

REST
Client

POST

/api/employees

EmployeeRequest

```
{  
  "firstName": "Juan",  
  "lastName": "Perez",  
  "email": "juan.perez@luv2code.com"  
}
```

Employee
REST
Controller

Response contains
new id / primary key

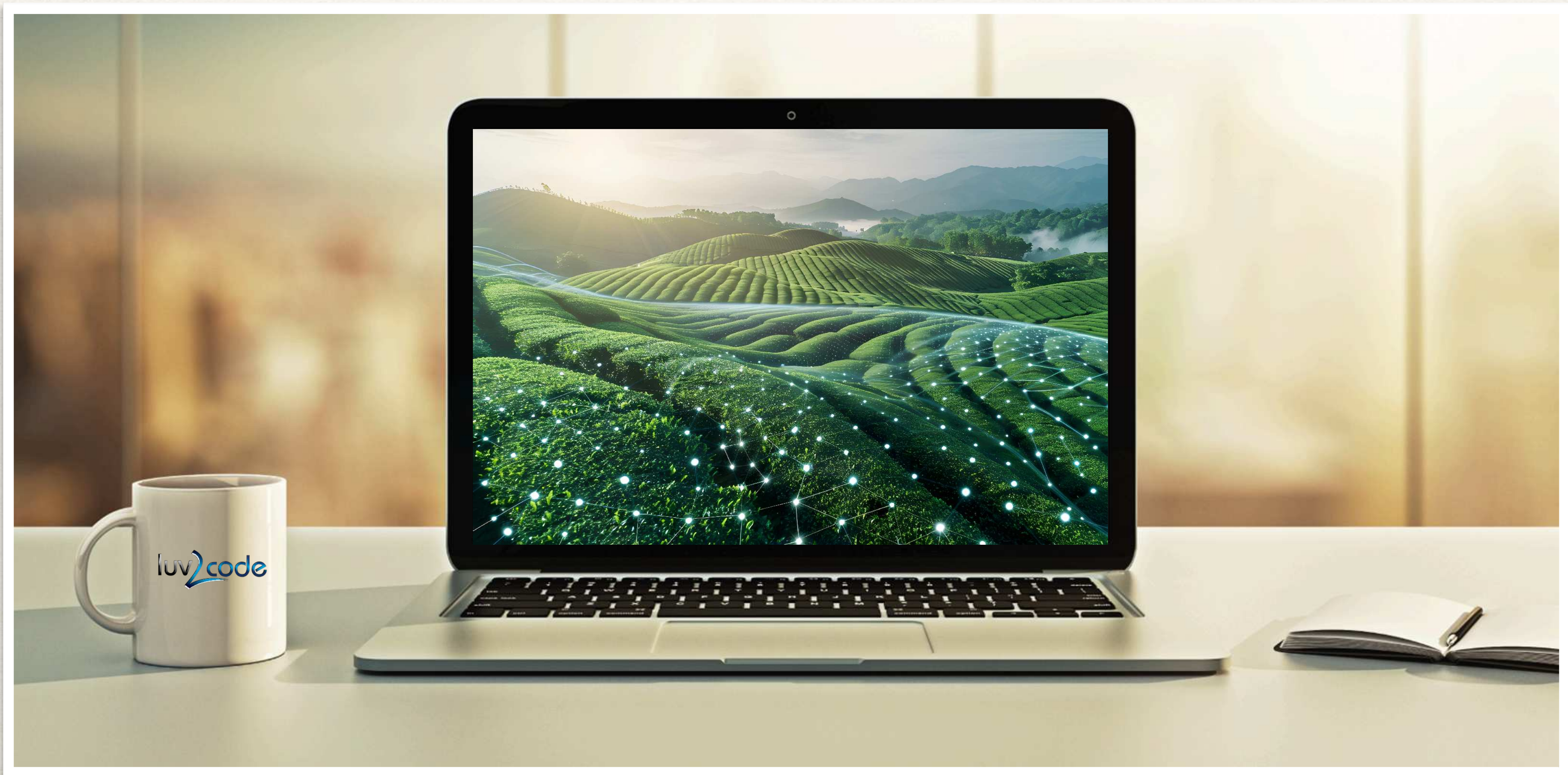
```
{  
  "id": 7,  
  "firstName": "Juan",  
  "lastName": "Perez",  
  "email": "juan.perez@luv2code.com"  
}
```

Employee

Sending JSON to Spring REST Controllers

- When sending JSON data to Spring REST Controllers
- For controller to process JSON data, need to set a HTTP request header
 - `Content-type: application/json`
- Need to configure REST client to send the correct HTTP request header
- Swagger handles this for us!

Rest Controller Methods - Update and Delete

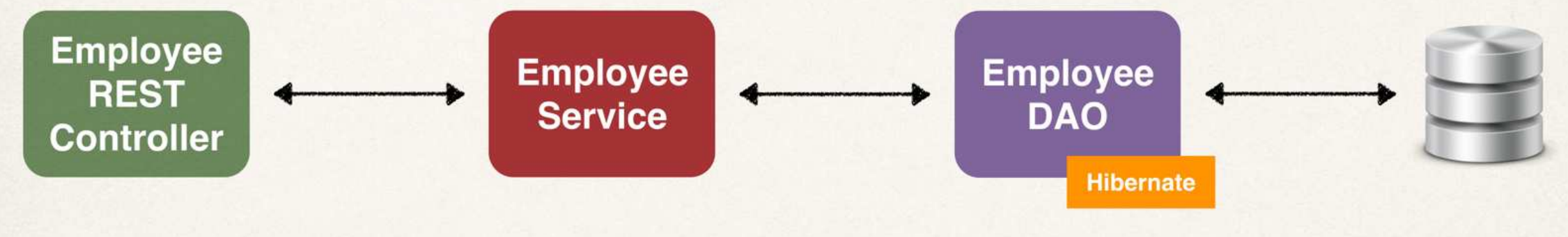


Development Process

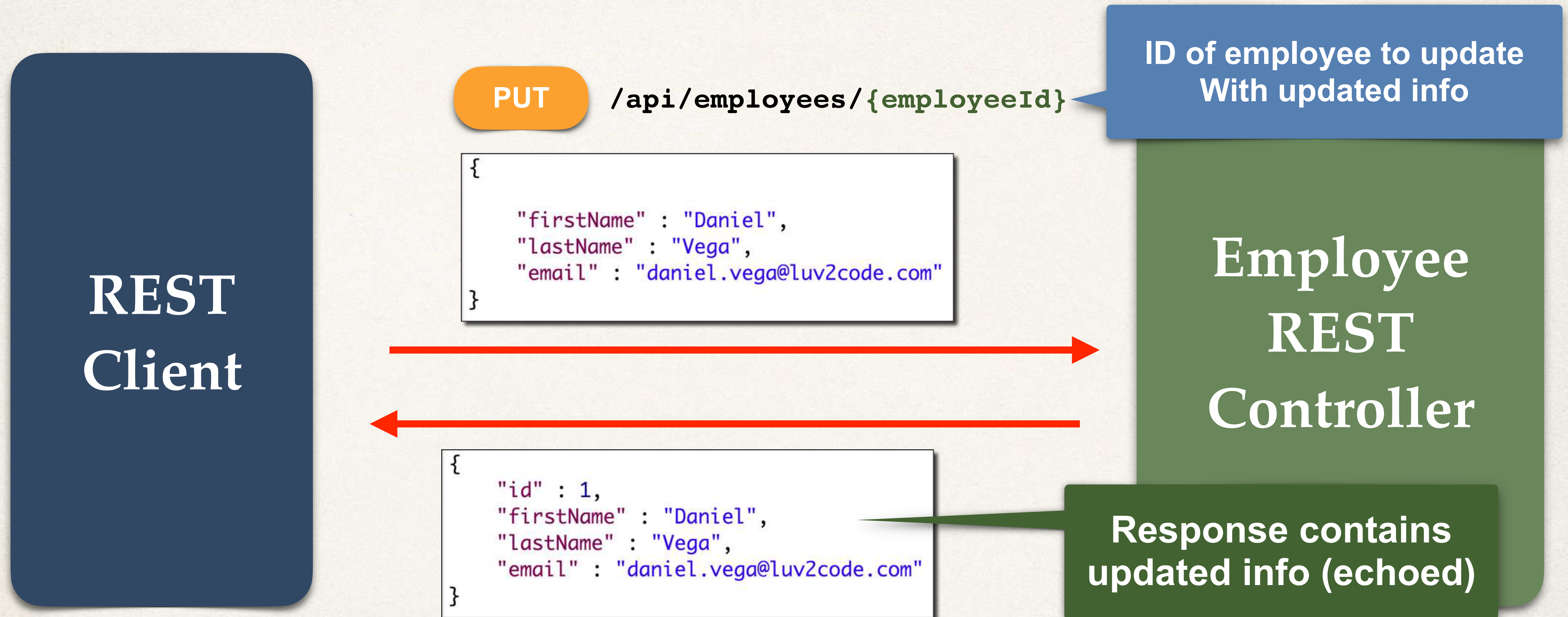
Step-By-Step

1. Create Spring Boot project using Spring Initializer
2. Set up Database Dev Environment
3. Get list of employees
4. Get single employee by ID
5. Add a new employee
6. Update an existing employee
7. Delete an existing employee

Rest Controller methods



Update Employee



Delete Employee

